An Efficient Method for Mining Closed Frequent High Average-Utility Occupancy Patterns

Thang La
Faculty of Information Technology, University of Science, Ho Chi Minh City Vietnam National University, Ho Chi Minh City, Vietnam
Faculty of Information Technology, Dalat University, Vietnam
thanglq@dlu.edu.vn

Khanh Tran
Faculty of Information Technology, Dalat University, Vietnam
khanhtnn@dlu.edu.vn

Anh Tran
FPT University, Danang, Vietnam
anhtn35@fe.edu.vn

Hai Duong*
Department of Mathematics and Computer Science, Dalat University, Vietnam
haidv@dlu.edu.vn

Bac Le
Faculty of Information Technology, University of Science, Ho Chi Minh City Vietnam National University, Ho Chi Minh City, Vietnam
lhbac@fit.hcmus.edu.vn

***Abstract*—**Discovering all Frequent High Average-Utility Occupancy Itemsets (FHAUOIs) in quantitative transaction databases has emerged as a promising direction in data mining. By integrating both frequency and average-utility occupancy, these itemsets are better aligned with real-world applications, as they not only represent the preferences of the majority of users but also contribute significantly to the utility of the transactions in which they appear. However, the set of all discovered FHAUOIs can be extremely large, especially when dealing with dense or large-scale datasets, or when low minimum threshold values are used. This can make it difficult for users to interpret and utilize the results effectively. To address this challenge, this paper proposes a novel algorithm called C-FHAUOIM for mining Closed FHAUOIs (CFHAUOIs). The algorithm leverages a newly designed upper bound on average-utility occupancy to efficiently reduce the database by eliminating low-average-utility occupancy items. In addition, it introduces a novel weak lower bound, along with associated pruning strategies, to effectively discard non-closed FHAUOIs early in the mining process. These theoretical advances are integrated into an efficient algorithm for discovering CFHAUOIs. Extensive experiments on several real-world and synthetic quantitative databases demonstrate that the proposed algorithm significantly outperforms the baseline approach in terms of runtime, memory consumption, and scalability.

Keywords*—Closed itemset, high average-utility occupancy itemset, upper bound, weak upper bound, pruning strategy.*

I. INTRODUCTION

High Utility Itemset Mining (HUIM) [1] and High Average-Utility Itemset Mining (HAUIM) [2] over Quantitative Transaction Databases (QTDBs) have recently emerged as prominent research areas, driven by their wide-ranging applications across domains such as identifying profitable product combinations, analyzing user behavior in online retail platforms, biomedical data analysis, and cross-marketing strategies [3][4][5]. In these problems, each item in a transaction is associated with both a quantity (e.g., number of units purchased) and a quality value (e.g., unit profit), which together determine the utility of that item.

HAUIM is generally considered more advantageous than HUIM because it employs a balanced evaluation metric based on average utility [2], which accounts for both the total utility and the length of the itemset. However, HAUIM over QTDBs faces a fundamental challenge: the average-utility function does not satisfy the anti-monotonicity property, a key characteristic commonly used to prune the search space in many pattern mining tasks. To overcome this limitation, researchers have introduced various upper bounds (UBs) that fully or partially preserve anti-monotonic behavior, enabling more effective pruning strategies. Over time, several tight UBs and weak upper bounds (WUBs) have been proposed, such as $mrau$ [8], $\overline{vmaub}$ [9] and $tvwaub$ [10].

Related work. While and HAUIM have contributed significantly to utility-based pattern mining, a critical limitation remains: the discovered patterns do not inherently reflect how much utility they contribute within each supporting transaction. In other words, these approaches lack a measure of utility representativeness at the transaction level.

To address this gap, the concept of utility occupancy (UO) was proposed [11][12]. UO quantifies the proportion of a pattern's utility relative to the total utility of the transaction in which it appears. This metric offers a finer-grained perspective by highlighting a pattern's significance and representativeness within individual transactions, rather than just its global utility or frequency. Based on the definition of UO, researchers proposed the problem of Frequent High Utility Occupancy Itemset Mining (FHUOIM) [13],[14],[15], which has proven effective in various real-world applications [13].

Building on utility occupancy, Kumar et al. [16] introduced Frequent High Average-Utility Occupancy Itemset Mining (FHAUOIM) by combining average-utility occupancy with support, offering a more balanced measure than FHUOIM. However, FHAUOIM can produce an overwhelming number of patterns, especially in large, dense QTDBs or with low thresholds, which makes it difficult for users to analyze the results and extract actionable insights, thus limiting the practical utility of the mining process. To address this, researchers mine concise representations such as closed and generator patterns [7][17], which reduce redundancy while preserving essential information; closed patterns, in particular, capture the most representative high-utility itemsets in each equivalence class, groups of itemsets sharing the same transaction coverage. In the context of FHUOIM, prior studies have proposed methods to mine both closed FHUOIs [18] and their generators [19]. While closed FHAUOIs are even more desirable than closed FHUOIs, no work has addressed the Closed FHAUOIM (CFHAUOIM) problem. This paper fills that gap by establishing its theoretical foundations and proposing an efficient mining algorithm.

Contributions. The key contributions of this paper are threefold. First, it proposes two novel bounds, an upper bound ($lubauo$) and a weak lower bound ($wlbauo$), along with

effective pruning strategies to significantly reduce the search space. Second, it presents an efficient algorithm, C-FHAUOIM, for mining closed FHAUOIs using the proposed theoretical results. Extensive experiments on real-world and synthetic datasets demonstrate that the proposed method consistently outperforms the baseline in terms of runtime, memory usage, and scalability.

The remainder of this paper is organized as follows. Section II provides the preliminary concepts. Section III introduces the proposed novel bounds, pruning strategies, and the C-FHAUOIM algorithm. Section IV presents the experimental evaluation and analysis. Finally, Section V concludes the paper and outlines potential directions for future work.

II. Preliminary Concepts and Problem Statement

This section introduces the fundamental concepts and notations relevant to mining frequent and high average utility occupancy itemsets. Let $\mathcal{A} \stackrel{\text{def}}{=} \left\{ a_j, j \in J \stackrel{\text{def}}{=} \{1, 2, \dots, M\} \right\}$ be a set consisting of a finite number of distinct items and A be a subset (itemset) of $\mathcal{A}$, $A \subseteq \mathcal{A}$. If A contains exactly k items, then it is referred to as a k-itemset, i.e. $k = |A|$. Moreover, assume that, without loss of generality, all items in each itemset are sorted according to a predefined total order relation $\prec$, e.g. the lexicographic order over items. A profit vector $\mathcal{P} \stackrel{\text{def}}{=} \left(p(a_j), j \in J \text{ and } p(a_j) > 0\right)$ indicates the external utility of all items in $\mathcal{A}$, where $p(a_j)$ denotes the relative importance of item a_j, such as its unit profit, price, or weight. A pair (a, q) is called a q-item, where $a \in \mathcal{A}$ and q is a positive integer representing the purchase quantity or internal utility of item a.

A quantitative database (QDB) $\mathcal{D}$ is a collection of transactions, $\mathcal{D} \stackrel{\text{def}}{=} \left\{t_i, i \in I \stackrel{\text{def}}{=} \{1, 2, \dots, N\}\right\}$, where each transaction t_i, defined as $t_i \stackrel{\text{def}}{=} \{(a_j, q_{ij}), j \in J_i\}$ including a set of q-items, is uniquely identified by a transaction identifier ($TID = i$), and J_i is an index subset of the J set, $J_i \subseteq J$. The utility of a q-item (a, q) is denoted and defined as $u(a, q) \stackrel{\text{def}}{=} q * p(a)$. In simpler terms, it represents the profit earned from selling item a when the transaction contains the q-item (a, q). To avoid redundant calculations of the utility u for q-items (a_j, q_{ij}) across transactions t_i of $\mathcal{D}$, these values are computed only once. The result is stored in a new corresponding QDB, called the integrated QDB according to $\mathcal{D}$ and $\mathcal{P}$, denoted and defined as $\mathcal{D}' \stackrel{\text{def}}{=} \{t_i \stackrel{\text{def}}{=} \{(a_j, q'_{ij}), j \in J_i\}, i \in I\}$, where $q'_{ij} = q_{ij} * p(a_j)$ if $(a_j, q_{ij}) \in t_i, \forall i \in I, j \in J_i$.

Definition 1 (*Operator ρ and support of an itemset* [18]).

a. For any non-empty itemset $A \subseteq \mathcal{A}$, let $\rho(A) \stackrel{\text{def}}{=} \{t \in \mathcal{D}' \mid t \supseteq A\}$ be the set of transactions in $\mathcal{D}'$ that contain A, where $t \supseteq A$ means that each item a_j of A must appear in a q-item (a_j, q_{ij}) of t (or t is called a supporting transaction of A). By convention, $\rho(\emptyset) \stackrel{\text{def}}{=} \mathcal{D}'$.

b. The support of an itemset A, denoted as $supp(A)$, is given by the number of transactions in $\mathcal{D}'$ containing A, i.e. $supp(A) = |\rho(A)|$.

c. The set of all non-empty itemset A that have at least one occurrence in the transactions of $\mathcal{D}'$ is denoted and defined as $\mathcal{IS} \stackrel{\text{def}}{=} \{A \subseteq \mathcal{A} \mid (A \neq \emptyset) \wedge (\rho(A) \neq \emptyset)\}$.

Table 1. The integrated QDB $\mathcal{D}'$.

TID	*Transaction (item, utility)*
t_1	$(a, 4), (b, 20), (c, 112), (d, 100), (e, 42), (f, 8)$
t_2	$(b, 8), (c, 4), (e, 240), (f, 16)$
t_3	$(b, 240), (e, 90), (f, 4)$
t_4	$(a, 10), (d, 20)$
t_5	$(a, 4), (b, 24), (c, 6), (d, 8), (e, 165), (f, 20)$

Running example. For example, consider a QDB $\mathcal{D}$ consisting of 6 transactions $t_1 = \{(a, 2), (b, 5), (c, 56), (d, 25)(e, 14), (f, 4)\}$, $t_2 = \{(b, 2), (c, 2), (e, 80), (f, 8)\}$, $t_3 = \{(b, 60), (e, 30), (f, 2)\}$, $t_4 = \{(a, 5), (d, 5)\}$, and $t_5 = \{(a, 2), (b, 6), (c, 3), (d, 2), (e, 55), (f, 10)\}$, and the profit vector $\mathcal{P} = (2; 4; 2; 4; 3; 2)$ of 6 items a, b, c, d, e and f. The integrated QDB $\mathcal{D}'$ according to $\mathcal{D}$ and $\mathcal{P}$ in Table1 will be used as the running example. It is observed that there are 2 transactions containing bc, $\rho(bc) = \{t_1, t_2, t_5\}$, $supp(bc) = 2$.

Definition 2 (*Transaction utility, utility* [3] *and average utility* [2] *of an itemset in a transaction*). Let us consider any transaction $t \in \mathcal{D}'$.

a. The utility of item a in t (i.e. $\exists (a, q) \in t$) and the transaction utility of t are respectively defined as $u(a, t) \stackrel{\text{def}}{=} q$ and $u(t) \stackrel{\text{def}}{=} \sum_{(a,q) \in t} u(a, t)$.

b. For any itemset A in t ($A \subseteq t$, i.e. all items of A appear in t), the *utility* and *average utility* of A in t are denoted and defined as $u(A, t) \stackrel{\text{def}}{=} \sum_{a \in A} u(a, t)$ and $au(A, t) \stackrel{\text{def}}{=} \frac{u(a,t)}{|A|}$, respectively, where $|A|$ denotes the length (or the number of items) of A.

Definition 3 (*Utility occupancy* [13], *average utility occupancy* [16] *of itemset*).

a. The *utility occupancy* of an itemset A in a supporting transaction $t \in \mathcal{D}'$, $A \subseteq t$, is defined as the *relative utility* of A in t compared to the transaction utility of t, given by $uo(A, t) \stackrel{\text{def}}{=} \frac{u(A,t)}{u(t)}$.

b. The average of the *utility occupancies* of A across all its supporting transactions $t \in \rho(A)$ is called the *utility occupancy* of A in the entire $\mathcal{D}'$, denoted and defined as $uo(A) \stackrel{\text{def}}{=} \frac{1}{supp(A)} \sum_{t \in \rho(A)} uo(A, t)$.

c. The *average utility occupancy* of A (in $\mathcal{D}'$) is denoted and defined as $auo(A) \stackrel{\text{def}}{=} \frac{uo(A)}{|A|}$. If we define $auo(A, t) \stackrel{\text{def}}{=} \frac{uo(A,t)}{|A|} = \frac{au(A,t)}{u(t)}$, then $auo(A)$ can be represented in another form as $auo(A) = \frac{1}{supp(A)} \sum_{t \in \rho(A)} auo(A, t)$.

Obviously, for any $A \in \mathcal{IS}$, since $auo(A, t) \in (0; 1], \forall t \in \rho(A)$, we always have $auo(A) \in (0; 1]$.

Example 1. For example, consider itemset $A = bc$, the utility and average utility of A in t_1 are respectively $u(A, t_1) = 20 + 112 = 132$ and $au(A, t_1) = \frac{132}{2} = 66$. Since the transaction utility of t_1 is $u(t_1) = 286$, the utility occupancy of A in t_1 is $uo(A, t_1) = \frac{u(A,t_1)}{u(t_1)} = \frac{132}{286} = 0.46$. Thus, the average utility

occupancy of A (with length 2) in t_1, $auo(A, t_1) = \frac{0.46}{2} = 0.23$. Similarly, by considering all transactions containing A, $\rho(A) = \{t_1, t_2, t_5\}$, the average utility occupancy of A is determined as follows: $auo(A) = \left(0.23 + \left(\frac{12}{268} + \frac{30}{227}\right)/2\right)/3 = 0.106$.

Definition 4 (*Frequent high average-utility occupancy itemset*). Given two user-specified positive *minimum average utility occupancy* and *support* thresholds, denoted respectively as mo and ms, an itemset A is called a *high average-utility occupancy* itemset (HAUOI) if its average-utility occupancy in $\mathcal{D}$ is not less than mo, $auo(A) \geq mo$. Otherwise, $auo(A) < mo$, A is called a *low average-utility occupancy* itemset (LAUOI). An HAUOI A is said to be a *frequent HAUOI* (FHAUOI) if its support is not less than ms, $supp(A) \geq ms$. The problem of FHAUOI mining ($\mathcal{FHAUOIM}$) is to discover the set $\mathcal{FHAUOI} \stackrel{\text{def}}{=} \{A \in \mathcal{IS} \mid auo(A) \geq mo \wedge supp(A) \geq ms\}$ of all FHAUOIs.

Example 2. Consider itemset $A = bc$ in Example 1, $ms = 1$ and $mo = 0.1$. Since $auo(A) = 0.106 > mo$ and $supp(A) = 3 > ms$, the A itemset is an FHAUOI.

Definition 5 (*Set $\mathcal{CFHAUOI}$*). Let A be an FHAUOI, $A \in \mathcal{FHAUOI}$. A is called a closed frequent high average-utility occupancy itemset (CFHAUOI) if there exists no FHAUOI being a proper superset of A and has the same support. Then, let $\mathcal{CFHAUOI}$ denote the set of all CFHAUOIs, defined as

$$\mathcal{CFHAUOI} \stackrel{\text{def}}{=} \{A \in \mathcal{FHAUOI} \mid \nexists B \in \mathcal{FHAUOI}: B \supset A \wedge supp(B) = supp(A)\}.$$

Problem Definition. Given a QDB $\mathcal{D}'$ and user-defined positive thresholds for minimum average utility occupancy mo and minimum support ms, the task of closed frequent high average utility occupancy itemset mining ($\mathcal{CFHAUOIM}$) involve identifying the sets $\mathcal{CFHAUOI}$.

Example 3. For $ms = 2$ and $mo = 0.1$, the result set $\mathcal{CFHAUOI}$ discovered from the QDB $\mathcal{D}'$ in Table 1 is. $\mathcal{CFHAUOI} = \{abcdef, ad, bcef, bef\}$.

Definition 6 (*Forward extension of an itemset* [9]). Assume that all items in $\mathcal{A}$ are sorted according to the total order relation $\prec$. For any item y and two itemsets A and B, let us denote $A \prec y$ iff $x \prec y$, $\forall x \in A$, or $A \prec B$ iff $x \prec y$, $\forall x \in A, \forall y \in B$. When $A \cap B = \emptyset$, $A \cup B$ is denoted as $A + B$. If $A \cap B = \emptyset$ and $A \prec B$, then the notation $A + B$ is replaced by the notation $A \oplus B$, and the itemset $S = A \oplus B$ is said to be a *forward extension* (FE), or simply an extension, of A with B.

III. Mining CFHAUOIs

A. Pruning of low average utility occupancy itemsets

To efficiently reduce the search space, it is essential to leverage the *anti-monotonicity ($\mathcal{AM}$)* inherent in the support function, along with upper bounds (UB) on auo. For instance, if an itemset A is infrequent then all its super-itemsets are also infrequent, and as a result, the entire branch $Br(A)$, which contains only infrequent extensions of A, can be pruned. A function ub is a UB on auo if it satisfies $auo(C) \leq ub(C)$, $\forall C \in \mathcal{IS}$.

It is well-known that $supp$ is $\mathcal{AM}$. However, since the $\mathcal{AM}$ property does not hold for auo, it is necessary to design $\mathcal{AM}$ UBs on auo. Inspired by the $\mathcal{AM}$ UB $auoub$ on auo [16], which represents the maximum item utility occupancy, we generalize it and develop a tighter new $\mathcal{AM}$ UB, named $lubauo$. Its value is the average of the top-k largest item utility occupancies. The $\mathcal{AM}$ property of $supp$ and $lubauo$ can be utilized to remove *infrequent* or *LAUO branches* from the prefix search tree.

Definition 7 (*Upper bound $lubauo$ on auo*). For any frequent itemset $A \in \mathcal{FI}$ and transaction $t \in \rho(A)$, after sorting all item utilities of t in descending order, we obtain the series $T = (a_1, a_2, \ldots, a_n)$, where $n = |t|$.

a. The *k-largest* average utility of t is defined as $lau(t, k) \stackrel{\text{def}}{=} \frac{1}{k}\sum_{i=1..k,\, a_i \in T} a_i$ for $1 \leq k \leq n$, and $lubauo(A, t) \stackrel{\text{def}}{=} \frac{lau(t, |A|)}{u(t)}$.

b. An upper bound (UB) on auo, called $lubauo$ (*largest average utility occupancy upper bound*), is defined as

$$lubauo(A) \stackrel{\text{def}}{=} \frac{1}{ms}\sum_{i=1..ms,\, v_i \in V} v_i,$$

where $V = \{v_i, i = 1..|\rho(A)|\}$ that is obtained by sorting the series $\{lubauo(A, t) \mid t \in \rho(A)\}$ in descending order.

Example 4. For $ms = 2$, consider itemset $A = bc$ with $supp(A) = 3$, $length(A) = 2$ and $\rho(A) = \{t_1, t_2, t_5\}$. The computation of the new UB $lubauo(A)$ is as follows. First, for transaction t_1, we have $lubauo(A, t_1) = (112 + 100)/2/286 = 0.37$ because 112 and 100 are the two largest utilities of t_1. Similarly, for the two remaining transactions t_2 and t_5, $lubauo(A, t_2) = (240 + 16)/2/268 = 0.48$ and $lubauo(A, t_5) = (165 + 24)/2/227 = 0.42$. Since $lubauo(A, t_2) > lubauo(A, t_5) > lubauo(A, t_1)$, we have $lubauo(A) = (0.48 + 0.42)/2 = 0.45$.

Proposition 1 (*The $\mathcal{AM}$ properties of $supp$ and UB $lubauo$*).

a. $lubauo$ is a UB on auo, i.e. $auo(A) \leq lubauo(A), \forall A$

b. $lubauo$ is $\mathcal{AM}$, meaning $lubauo(B) \leq lubauo(A)$ for any super-itemset B of A in $\mathcal{FI}$, $B \supseteq A$.

Example 5. For $A = bc$ and $mo = 0.5$, the $Br(A)$ branch can be eliminated utilizing the new UB $lubauo$ since $lubauo(A) = 0.45 < mo$. For example, $auo(A) = 0.106 < lubauo(A) < mo$. Hence, the tighter UB $lubauo$ will be used in the pruning strategies, as stated in Theorem 1 below.

Let $PC(A) \stackrel{\text{def}}{=} (supp(A) < ms) \vee (lubauo(A) < mo)$ represent the *pruning condition* used to check whether A is an infrequent or LAUO itemset. Since the functions $supp$ and $lubauo$ are $\mathcal{AM}$, the necessary condition for any FE of A to be an FHAUOI is $not(PC(A))$. Based on the definition of PC, the following theorem is obtained.

Theorem 1 (*Pruning strategy 1 based on $supp$ and $lubauo$ – PS1*). For any item $y \in \mathcal{A}$, if $PC(y)$ is true, then this irrelevant item can be safely removed from the database $\mathcal{D}'$. Furthermore, for any itemset A, if $PC(A)$ holds, the entire branch $Br(A)$ can also be pruned from the search tree.

B. Pruning of non-closed FHAUOIs

For brevity, for any itemset B, if $B \notin \mathcal{FHAUOI}$ (the set of all FHAUOIs) or B is not a CFHAUOI (Closed frequent high average-utility occupancy itemsets), then B is referred to as a *non-closed FHAOU* ($NCFHAOU$) itemset. Similarly, *if the* $Br(B)$ *branch, which includes* B *and all its FEs, has no CFHAUOIs, it is referred to as an* $NCFHAUO$ *branch*.

Note that the above UB targets fast elimination of LAUOIs, but FHAUOI mining can still yield many results, especially in large QDBs or with low mo and ms. Extracting CFHAUOIs from all FHAUOIs is challenging due to costly inclusion checks. This motivates designing weak lower bounds (WLB) on auo to enable early pruning of non-closed FHAUOIs and unpromising branches.

Definition 8 (*WLB and LB on* auo). For any transaction and itemset A, a function $wlb(A,t)$ is called a WLB on $auo(A,t)$ if $wlb(A,t) \leq auo(C,t)$ for any *proper FE* C of A. Similarly, wlb is referred to as a WLB on auo if $wlb(A) \leq auo(C)$ for any *proper FE* C of A.

Definition 9 (*WLB* $wlbauo$ *on* auo *in each transaction*). For any itemset A and transaction $t \in \delta(A)$. A WLB on $auo(A,t)$, denoted $wlbauo(A,t)$, is defined as follows.

$$wlbauo(A,t) = \frac{sau(A,t)}{u(t)},$$

where $sau(A,t)$ is the smallest average utility of proper forward extensions of A in t, which is defined as

$$sau(A,t) = \begin{cases} f(r), & if\ mu(A,t) \leq au(A,t) \\ f(1), & if\ mu(A,t) > au(A,t) \end{cases},$$

where $mu(A,t) = min\{u(a,t) \mid (a,q) \in rem(A,t)\}$, $r = |rem(A,t)|$, $l = length(A)$ and $f(k) \stackrel{\text{def}}{=} \frac{l.au(A,t) + k.mu(A,t)}{l+k}$ is an intermediate value between $au(A,t)$ and $mu(A,t)$, $\forall k = 1 \ldots r$.

By using $wlbauo(A,t)$, we design a new WLB on auo as follows. For any strictly frequent itemset A, i.e. $|\delta(A)| \geq ms$, we sort the series $\{wlbauo(A,t) \mid t \in \delta(A)\}$ in ascending order to create the new series $T \stackrel{\text{def}}{=} \{t_i, i = 1..n\}$, where $supp(A) \geq n \stackrel{\text{def}}{=} |\delta(A)| \geq ms$ since $\delta(A) \subseteq \rho(A)$.

Definition 10 (*WLB on* auo). For any itemset A such that $|\delta(A)| \geq ms$, a novel WLB on auo is denoted and defined as $wlbauo(A) = \frac{1}{ms}\sum_{i=1..ms} t_i$.

Proposition 2 (*Properties of WLB on* auo). For any itemset A, transaction $t \in \delta(A)$, and *proper* FE $B = A \oplus C$ of A with $C \neq \emptyset$ then

a) $wlbauo(A,t) \leq auo(B,t)$.

b) $wlbauo$ is a WLB on au, i.e. $wlbauo(A) \leq auo(B)$.

Example 6. For $ms = 2$, consider itemset $A = bc$, $l = length(A) = 2$, and $\delta(A) = \{t_1, t_2, t_5\}$. For transaction t_5, then $mu(A,t_5) = 8$, $r = 3$ and $au(A,t_5) = (24+6)/2 = 15$. Since $mu(A,t_1) < au(A,t_5)$, so $sau(A,t_5) = f(r) = f(3) = \frac{2*15+3*8}{2+3} = 10.8$. Thus, $wlbauo(A,t_5) = \frac{10.8}{227} = 0.048$. For t_1, since $mu(A,t_1) = 8 < au(A,t_1) = 66$, so $sau(A,t_1) = f(3) = \frac{2*66+3*8}{2+3} = 31.2$, and $wlbauo(A,t_1) = \frac{31.2}{286} = 0.109$. Similarly, for t_2, since $mu(A,t_2) = 16 > au(A,t_2) = 6$, so $sau(A,t_2) = f(1) = \frac{2*6+16}{2+1} = 9.33$, and $wlbauo(A,t_2) = \frac{9.33}{268} = 0.035$. After sorting the series $\{wlbauo(A,t) \mid t \in \delta(A)\} = \{0.109, 0.035, 0.048\}$ in ascending order, the new series $T = \{0.035, 0.048, 0.109\}$ is obtained. Then, $wlbauo(A) = (0.035 + 0.048)/2 = 0.0415$. For the proper FE B of A, $B = A \oplus e = bce$, then $auo(B) = 0.608 > wlbauo(A)$.

Based on the proposed WLB, $wlbauo$, this paper introduces *the first novel pruning strategy* aimed at early elimination of $NCFHAUO$ branches in the prefix search tree. This strategy leverages itemsets already generated in the $\mathcal{CFHAUOI}$ set and is presented as follows.

Theorem 2 (*Pruning Strategy 2 – PS2*). For the current itemset C being examined when constructing the $\mathcal{CFHAUOI}$ set, if there is a CFHAUOI $P \in \mathcal{CFHAUOI}$ such that $P \supset C$, $lastItem(P) = lastItem(C)$, and $supp(C) = supp(P)$, then if $wlbauo\ (P) \geq mo$, the entire $NCFHAUO$ proper branch $pBr(C)$ is pruned early. Additionally, if $auo(P) \geq mo$ or $C \notin \mathcal{FHAUOI}$, the $NCFHAUO$ node C is also eliminated.

Corollary 1 (*Pruning Strategy 3 – PS3*). For any two nodes $C_1 = D \oplus x$ and $C_2 = D \oplus y$ having the same parent in the prefix tree such that $y \succ x$, when extending C_1 with item y to have the larger itemset $P = C_1 \oplus y$, if $supp(P) = supp(C_2)$ and $wlbauo(P) \geq mo$, then the whole $NCFHAUO$ $pBr(C_2)$ proper branch is safely eliminated early. Additionally, the $NCFHAUO$ C_2 node is also discarded, if $auo(P) \geq mo$ or $C_2 \notin \mathcal{FHAUOI}$.

Note that, due to space limitations, the proofs of the theorems and propositions are omitted from this paper.

C. Proposed C-FHAUOIM Algorithm

Based on the proposed theoretical results, this subsection introduces a novel algorithm named C-FHAUOIM to efficiently discover the set $\mathcal{CFHAUOI}$. The algorithm is developed to address the challenges of mining this set. To effectively implement the suggested algorithm, we utilize the NVUV-list (or Utility-List) data structure presented in [9].

C-FHAUOIM ($\mathcal{D}'$, mo, ms)
Input: the QDB $\mathcal{D}'$, mo and ms.
Output: the set $\mathcal{CFHAUOI}$.
1. **Scan** $\mathcal{D}'$ to calculate the support and WUB *lubauo* of all items, and determine the set $I \stackrel{\text{def}}{=} \{x \in \mathcal{A} \mid lubauo(x) \geq mo \wedge supp(x) \geq ms\}$; *//Pruning strategy 1*
2. **Remove** all *irrelevant* items in $\mathcal{A} \backslash I$ from $\mathcal{D}'$; *// PS 1*
3. **Sort** I in ascending order of item support;
4. **Scan** $\mathcal{D}'$ again to construct the structure NVUV-list for all *relevant* items of I;
5. $\mathcal{CFHAUOI} := \emptyset$;
6. **for each** $x \in I$ **do** {
7. **Mine-CI**(x, mo, ms, $\mathcal{CFHAUOI}$);
8. }
9. **return** $\mathcal{CFHAUOI}$;

Fig. 1. Algorithm C-FHAUOIM

The pseudo-code of the proposed algorithm is depicted in Fig. 1, with the objective of mining the $\mathcal{CFHAUOI}$. It takes $\mathcal{D}'$, mo and ms as input. C-FHAUOIM initially scans $\mathcal{D}'$ to

calculate the $lubauo$ and support values for items in $\mathcal{A}$ and determine the I set (line 1). Then, C-FHAUOIM removes irrelevant items from $\mathcal{A}\backslash I$ in $\mathcal{D}'$ to minimize the candidate set for extension (line 2). After sorting the I set in ascending order of item support, the algorithm scans the QDB $\mathcal{D}'$ again to construct the structure NVUV-list for items in I (lines 3-4). Then, with each item x in I, the Mine-CI procedure is invoked at line 7 to discover CFHAUOIs in the $pBr(x)$ branch. Finally, C-FHAUOIM offers the $\mathcal{CFHAUOI}$ set as its output (line 9).

Mine-CI (C, mo, ms, $\mathcal{CFHAUOI}$)
Input: the itemset C, mo and ms thresholds, and set $\mathcal{CFHAUOI}$.
Output: $\mathcal{CFHAUI}$ is updated.
1. **if** ($supp(C) < ms$) **return;**
2. **for each** $P \in \mathcal{CFHAUOI}$ **and** $P \supset C$ **do** {
3. **NCFHAUO-Branch-Pruning** (C, P); // *Theorem 2*
4. **If** ($C.Prune_NC_pBr = true$) **then**
5. **break;**
6. }
7. **if** ($auo(C) \geq mo$) **then** {
8. **if** ($C.Prune_NC_Node = false$) **then**
9. **Update-Closed-FHAUOI** (C, $\mathcal{CFHAUOI}$);
10. }
11. **if** ($C.Prune_NC_pBr = true$) // *Pruning strategies 2-3*
12. **return**;
13. $x = lastItem(C)$; $D = prefix(C)$; $newI := \emptyset$;
14. **for each** $y \in I$ such that $y \succ x$) **do** {
15. **if** ($lubauo(Cy) \geq mo$ **and** $supp(Cy) \geq ms$) **then** //*PS 1*
16. $newI := newI \cup \{y\}$;
17. $Exts_of_C = \emptyset$;
18. **for each** $y \in newI$ **do** {
19. **Add** $P = Cy$ to $Exts_of_C$ **and** construct NVUV-list of P;
20. **NCFHAUO-Branch-Pruning** (Dy, P); // *PS3*
21. }
22. **for each** $P \in Exts_of_C$ **do**
23. **Mine-CI**(P, mo, ms, $\mathcal{CFHAUOI}$);

Fig. 2. Procedure Search-FHAUOS.

The Mine-CI procedure depicted in Fig. 2 first verifies if $supp(C)$ is less than ms. If yes, it stops immediately (line 1). Then, it checks if the node C or the $pBr(C)$ proper branch is pruned by Pruning Strategy 3 in Theorem 5 or not (line 2-6). If yes, the fields $Prune_NC_Node$ and/or $Prune_NC_pBr$ of itemset C are marked to prune in subsequent steps. Next, if C is an FHAUOI ($auo(C) \geq mo$) and the value of its field $Prune_NC_Node$ is false (lines 7-8), then the procedure Update-Closed-FHAUOI is called at lines 9-10 to update the $\mathcal{CFHAUOI}$ set. After that, Mine-CI evaluates whether any of the conditions outlined at line 12 are met. If affirmative, the $pBr(C)$ is pruned prematurely (line 13). In the case of negative, a new set of items, $newI$, is determined for expanding C (line 17), utilizing the criteria specified at line 16. Subsequently, each item from $newI$ is added to extend the itemset C, generating a new itemset $P = Cy$, and establishing the NVUV-list structure of P at line 20. The NCFHAUO-Branch-Pruning procedure is invoked to identify $NCFHAUO$ branches and nodes for subsequent pruning steps (line 21). Ultimately, Mine-CI calls itself recursively to explore larger CFHAUOIs.

IV. Experimental Evaluation

The experiments were conducted on 11th Gen Intel(R) Core (TM) i5–1135G7 @ 2.4 GHz (8 CPUs) with 12.288 GB RAM, running Windows 10 Home 64 bit. All comparison algorithms were implemented in Java SE 8 and executed using the NetBeans IDE. The evaluation was carried out on two QSDBs, including one real-world dataset BMS [21] and one synthetic dataset T20I4N600D70K, where T20I4N600D70K is dense and BMS is sparse. The characteristics of the QSDBs are shown in Table 2. Utilities were randomly generated using the IBM Quest data generator (obtained from [21]). To the best of our knowledge, no algorithm has been proposed for discovering CFHAUOIs to date. Therefore, we compare the performance of the C-FHAUOIM algorithm with a Baseline algorithm that does not utilize the novel theoretical results proposed this paper.

Table 2. Characteristics of databases

Database	#Tran	#Item	Avg. length	Data type
BMS	59,602	497	2.5	Real-life
T20I4N600D70K	70,000	600	20	Synthetic

A. Influence of the parameter mo

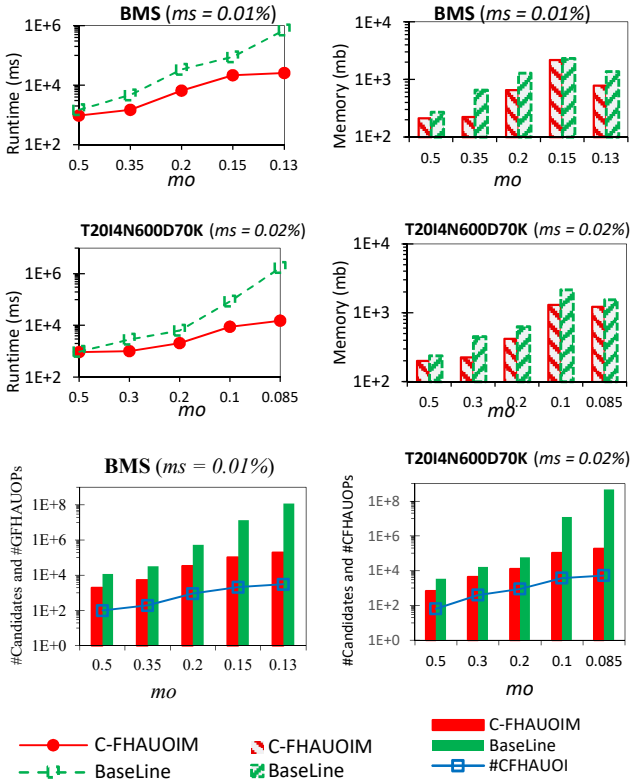

Fig. 3. The influence of varying the *mo* threshold.

Firstly, we examine the impact of *mo* on the performance of the algorithms using the QSDBs listed in Table 3, with results shown in Fig. 3. As *mo* decreases, execution time, memory usage, and the number of candidates grow substantially, since more patterns satisfy the threshold. The figure also shows that C-FHAUOIM consistently outperforms the Baseline in all three metrics. For example, on the QTDBs BMS and T20I4N600D70K, C-FHAUOIM is approximately 15 times and 66 times faster, respectively. This advantage stems from its novel strategies and the proposed bounds, which frequently satisfy pruning conditions and effectively shrink the search space. Moreover, C-FHAUOIM avoids frequent, costly inclusion-relationship checks, unlike the Baseline, further improving efficiency.

B. *Infuence of the parameter ms*

Next, we assess the effect of varying the *ms* parameter while keeping *mo* fixed, as shown in Fig. 4. Lower *ms* values lead to a sharp rise in execution time, memory usage, and #Candidates, since more patterns qualify for evaluation. Across all datasets, C-FHAUOIM clearly surpasses the Baseline in performance. For instance, on BMS and T20I4N600D70K, it achieves speedups of about 245 times and 39 times, respectively, thanks to its effective pruning strategies and proposed UB/WLB bounds that drastically reduce the search space and minimize costly inclusion checks.

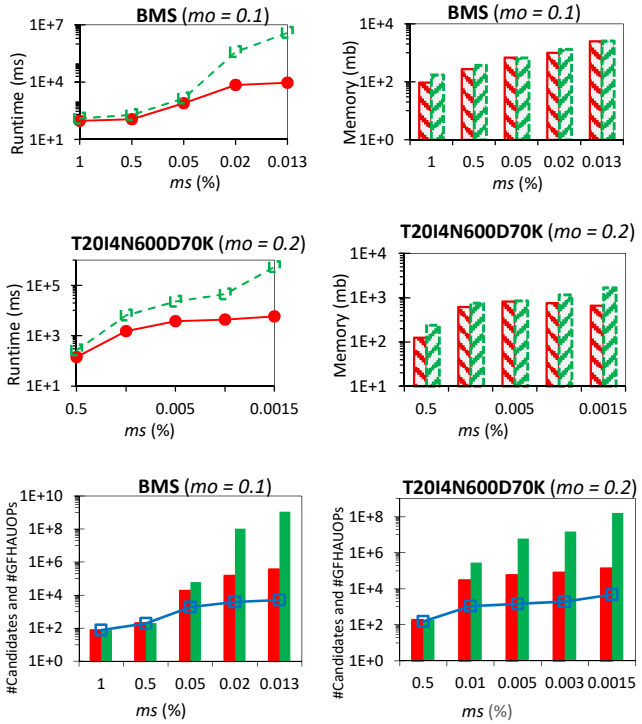

Fig. 4. The influence of varying the *ms* threshold.

In summary, both *mo* and *ms* have a substantial impact on runtime, memory usage, and the number of candidates. In every setting, C-FHAUOIM consistently outperforms the Baseline by achieving faster runtimes and lower memory consumption. These improvements are largely due to its effective UB/WLB-based pruning strategies, which greatly reduce the search space and avoid costly inclusion checks.

V. Conclustion

This paper presents new theoretical contributions to the problem of mining CFHAUOIs. To address this problem efficiently, we propose a novel upper bound and a weak lower bound for the *auo* function, along with associated pruning strategies that significantly reduce the search space during mining. Building on these theoretical insights, we introduce C-FHAUOIM, an algorithm specifically designed to effectively identify CFHAUOIs. Experimental evaluations demonstrate that C-FHAUOIM consistently outperforms the Baseline algorithm in terms of runtime, memory usage, and candidate count. Looking ahead, we plan to integrate parallel processing techniques to further boost the efficiency of CFHAUOI mining, particularly for large-scale datasets.

ACKNOWLEDGMENT. This research is funded by Dalat University, Vietnam under decision number 1153/QĐ-ĐHĐL, 2024.

References

[1] H. Yao and H. J. Hamilton, "Mining itemset utilities from transaction databases," *Data Knowl Eng*, vol. 59, no. 3 SPEC. ISS., pp. 603–626, 2006.

[2] T.-P. Hong, C. H. Lee, and S. L. Wang, "Mining high average-utility itemsets," in *Proceedings of IEEE International Conference on Systems, Man and Cybernetics*, 2009, pp. 2526–2530.

[3] M. Liu and J. Qu, "Mining high utility itemsets without candidate generation," in *Proceedings of ACM International Conference on Information and Knowledge Management*, 2012, pp. 55–64.

[4] B. E. Shie, P. S. Yu, and V. S. Tseng, "Mining interesting user behavior patterns in mobile commerce environments," *Applied Intelligence*, vol. 38, no. 3, pp. 418–435, 2013.

[5] V. S. Tseng, B. E. Shie, C. W. Wu, and P. S. Yu, "Efficient algorithms for mining high utility itemsets from transactional databases," *IEEE Trans Knowl Data Eng*, vol. 25, no. 8, pp. 1772–1786, 2013.

[6] Y. Liu, L. Wang, L. Feng, and B. Jin, "Mining high utility itemsets based on pattern growth without candidate generation," *Math*, vol. 9, no. 1, pp. 1–22, 2021.

[7] H. Duong, T. Hoang, T. Tran, T. Truong, B. Le, and P. Fournier-Viger, "Efficient Algorithms for Mining Closed and Maximal High Utility Itemsets," *Knowl Based Syst*, vol. 257, p. 109921, 2022.

[8] H. Kim, U. Yun, Y. Baek, J. Kim, B. Vo, and E. Yoon, "Efficient list based mining of high average utility patterns with maximum average pruning strategies," *Inf Sci (N Y)*, vol. 543, pp. 85–105, 2021.

[9] T. Truong, H. Duong, B. Le, P. Fournier-Viger, and U. Yun, "Efficient high average-utility itemset mining using novel vertical weak upper-bounds," *Knowl Based Syst*, vol. 183, no. 1, p. 104847, 2019.

[10] T. Tran, H. Duong, T. Truong, and A. Tran, "High Average-Utility Itemset Mining with A Novel Vertical Weak Upper Bound," in *2023 RIVF International Conference on Computing and Communication Technologies (RIVF)*, Dec. 2023, pp. 342–347.

[11] L. Tang, L. Zhang, P. Luo, and M. Wang, "Incorporating occupancy into frequent pattern mining for high quality pattern recommendation," in *ACM International Conference Proceeding Series*, 2012, pp. 75–84.

[12] B. Shen, Z. Wen, Y. Zhao, D. Zhou, and W. Zheng, "OCEAN: Fast discovery of high utility occupancy itemsets," in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 2016, pp. 354–365.

[13] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, and P. S. Yu, "HUOPM: High-Utility Occupancy Pattern Mining," *IEEE Trans Cybern*, vol. 50, no. 3, pp. 1195–1208, Mar. 2020.

[14] H. Kim *et al.*, "Mining high occupancy patterns to analyze incremental data in intelligent systems," *ISA Trans*, vol. 131, pp. 460–475, 2022.

[15] L. T. T. Nguyen, T. Mai, G. H. Pham, U. Yun, and B. Vo, "An efficient method for mining high occupancy itemsets based on equivalence class and early pruning," *Knowl Based Syst*, vol. 267, p. 110441, 2023.

[16] M. J. K. Kumar and D. Rana, "HAUOPM: High Average Utility Occupancy Pattern Mining," *Arab J Sci Eng*, vol. 49, pp. 3397–3416, 2024.

[17] H. Duong, T. Truong, B. Le, and P. Fournier-Viger, "CG-FHAUI: an efficient algorithm for simultaneously mining succinct pattern sets of frequent high average utility itemsets," *Knowl Inf Syst*, vol. 66, pp. 5239–5280, 2024.

[18] H. Duong, H. Pham, T. Truong, and P. Fournier-Viger, "Efficient algorithms to mine concise representations of frequent high utility occupancy patterns," *Applied Intelligence*, vol. 54, pp. 4012–4042, Mar. 2024.

[19] D. Hai and T. Tin, "An Efficient Algorithm for Mining Frequent High Utility Occupancy Generators," in *CCIOT '24: Proceedings of the 2024 9th International Conference on Cloud Computing and Internet of Things*, Association for Computing Machinery (ACM), Jan. 2025, pp. 101–109.

[20] T. Truong, H. Duong, B. Le, and P. Fournier-Viger, "Efficient Vertical Mining of High Average-Utility Itemsets Based on Novel Upper-Bounds," *IEEE Trans Knowl Data Eng*, vol. 31, no. 2, pp. 301–314, 2019.

[21] P. Fournier-Viger *et al.*, "SPMF: a Java Open-Source Pattern Mining Library Version 2," *Machine Learning Research*, vol. 15, no. 1, pp. 3389–3393, 2014.